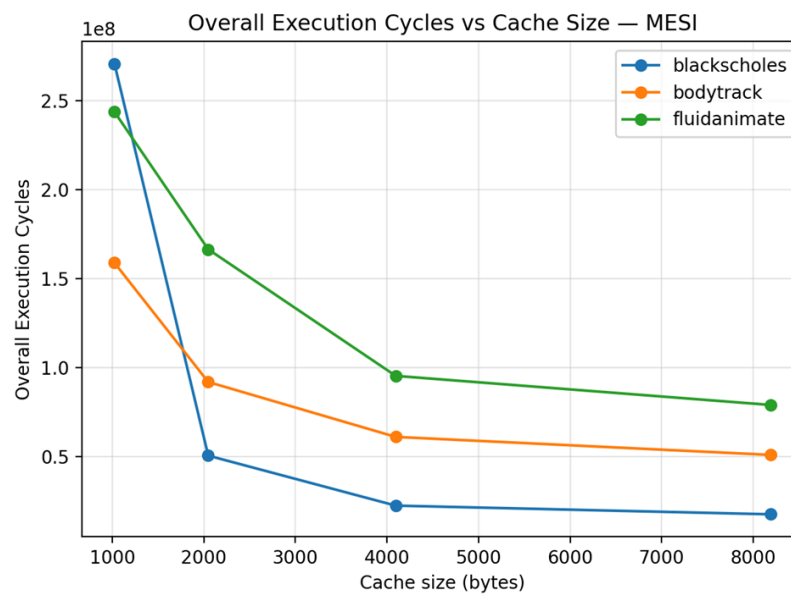
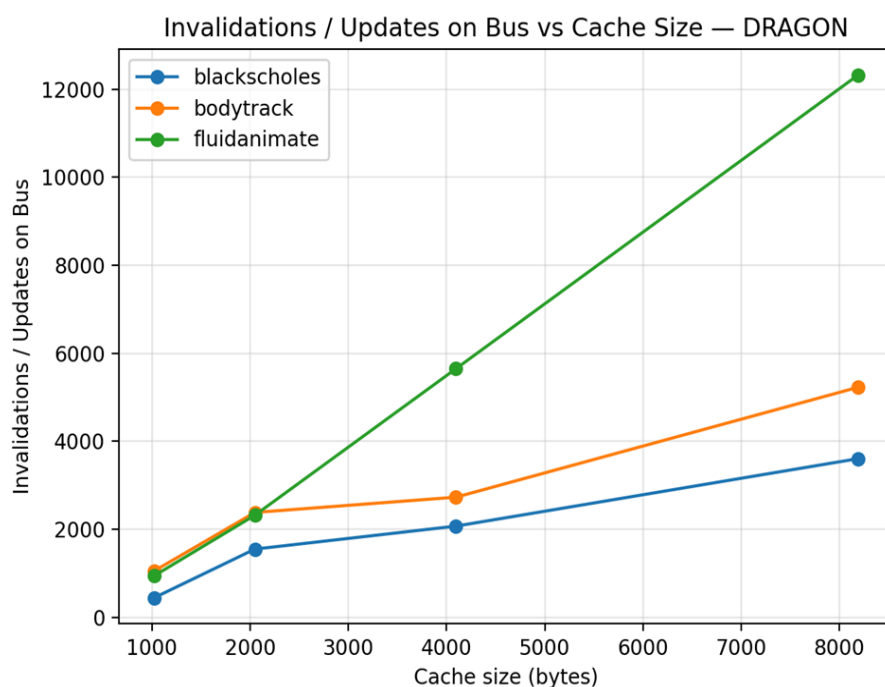


Quantitative Analysis

We perform our quantitative evaluation using the cache simulator on three PARSEC benchmarks (blackscholes, bodytrack, fluidanimate) under three coherence protocols: MESI, Dragon and an additional MOESI as an improvement implementation. For each protocol-benchmark pair, we systematically sweep several cache configurations by varying cache size, associativity and block size while keeping the remaining parameters fixed. The simulator reports overall execution cycles and a set of memory-system metrics (including bus data traffic and the number of coherence invalidations/updates), and all runs are captured into a single log file. We then parse this log into a CSV file and generate line plots, allowing us to compare how cache size and coherence protocol jointly affect performance and coherence overhead across the three workloads. Please see Appendix for more details.

1. Dataset



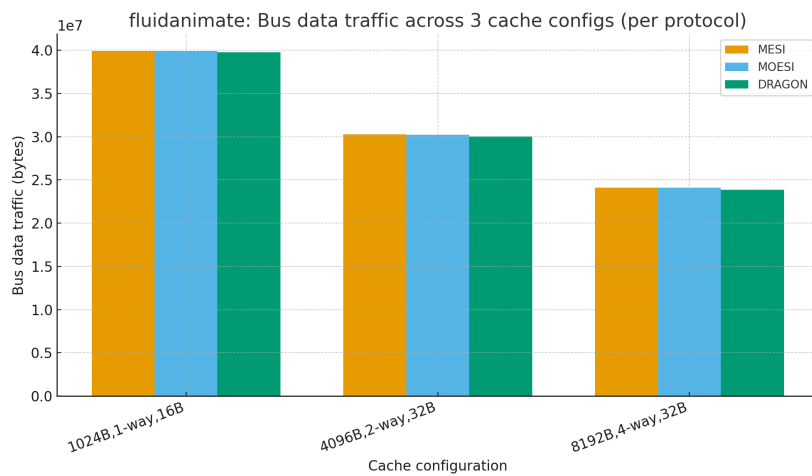
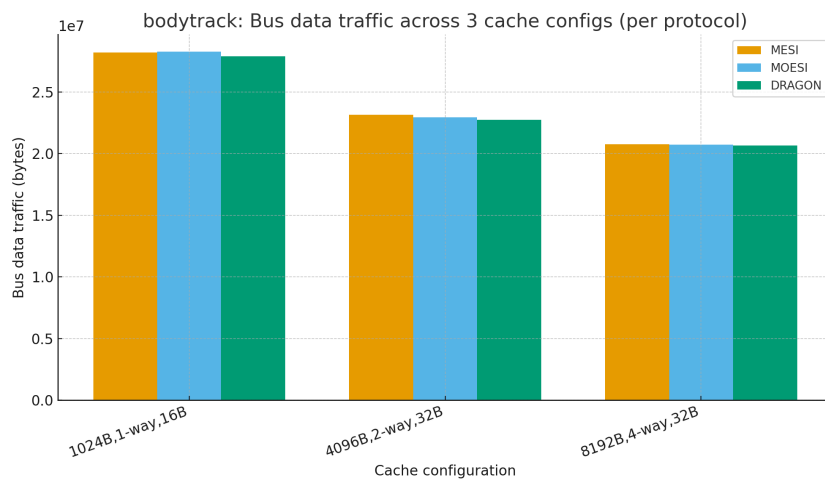
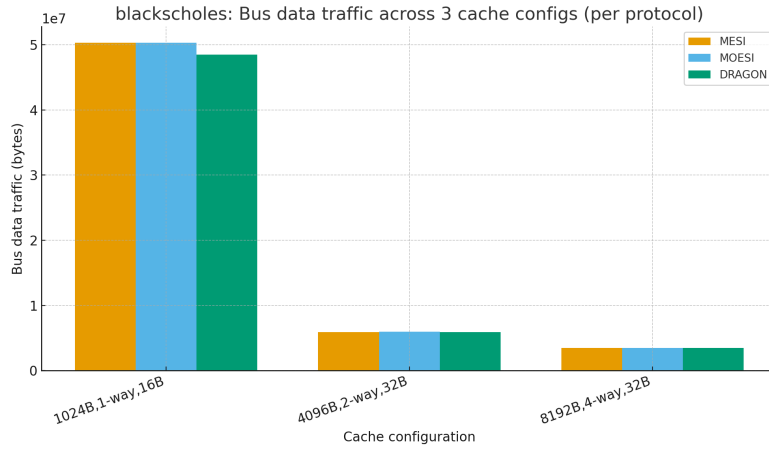


For **blackscholes**, the MESI overall-cycles figure shows its curve dropping much more steeply than the others as cache size increases, indicating that performance is highly sensitive to cache capacity. Once the cache is large enough, most of the working set fits and execution time falls sharply. In the DRAGON updates figure, blackscholes consistently has the lowest update count across all cache sizes, which means there is relatively little true sharing and most accesses are effectively private or read-mostly, so protocol choice has only a minor impact compared to cache size.

For **bodytrack**, the MESI overall-cycles figure shows a clear improvement with larger caches, but its curve is much flatter than **blackscholes**, so cache size matters, just not as dramatically. In the DRAGON updates figure, the bodytrack line sits above **blackscholes** at every cache size, reflecting more frequent writes to shared data and more inter-core communication. This explains why bus activity remains high even when the cache is scaled up and why different coherence protocols tend to perform similarly on this workload.

For **fluidanimate**, the MESI overall-cycles figure places its curve close to bodytrack's: execution time drops as cache size grows, giving a solid speedup, but again not on the same scale as **blackscholes**. In the DRAGON updates figure, **fluidanimate** typically has the highest and most rapidly increasing update count as cache size increases, which indicates that a large amount of data is shared and kept resident in multiple caches. Larger caches reduce misses and improve performance, but they also amplify the amount of coherence work needed to keep those shared lines consistent.

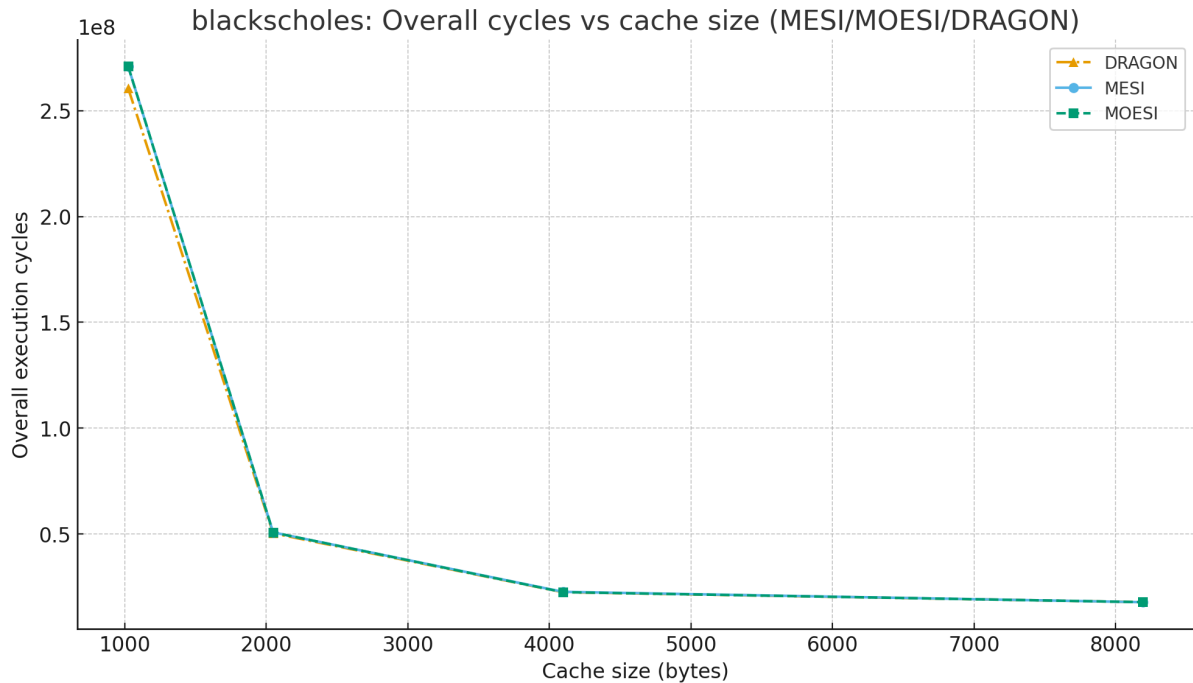
2 Effect of protocol on bus traffic



For bus traffic, the three-configuration bar charts show that increasing cache size (and associativity) from 1024B–1-way–16B to 4096B–2-way–32B and 8192B–4-way–32B reduces bus data traffic for all three protocols, but the relative differences between MESI, MOESI and DRAGON at each configuration are very small. In every benchmark, the three bars at a given configuration sit close together, with DRAGON usually a bit lower than MESI/MOESI and MOESI almost exactly overlapping MESI. This means that while a larger,

more associative cache clearly cuts total bus traffic, the choice of coherence protocol only shifts the bus usage by a few percent and does not fundamentally change the traffic pattern.

3 Cache size effect on protocol



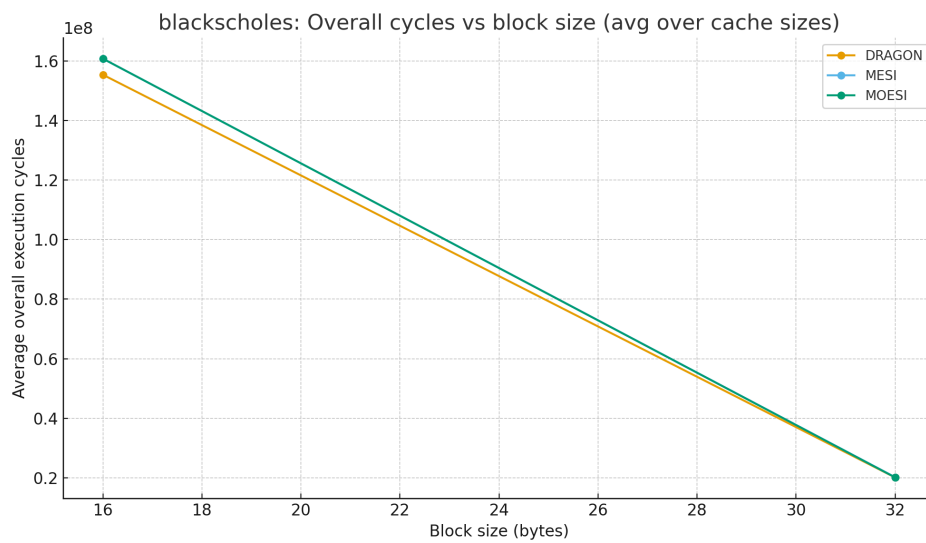
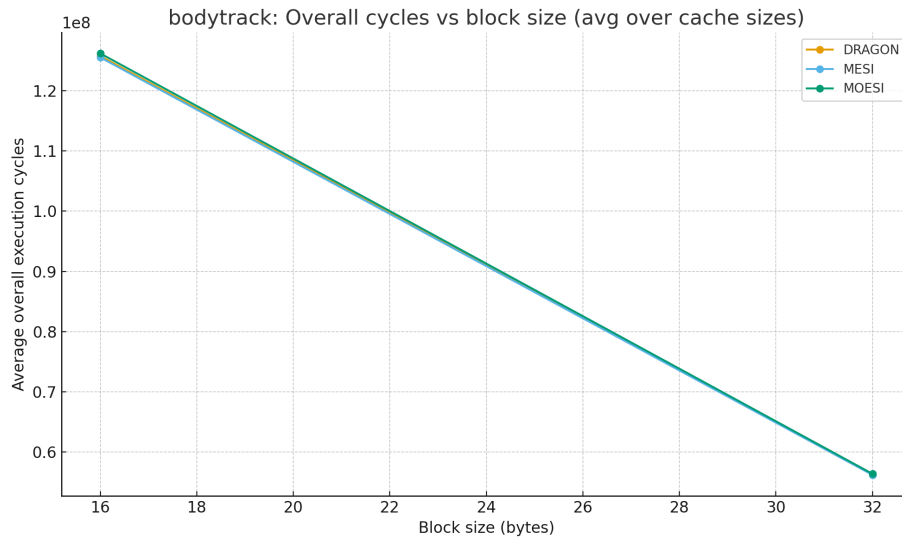
Notes: MESI and MOESI in the graph above is overlap so it may be hard to see

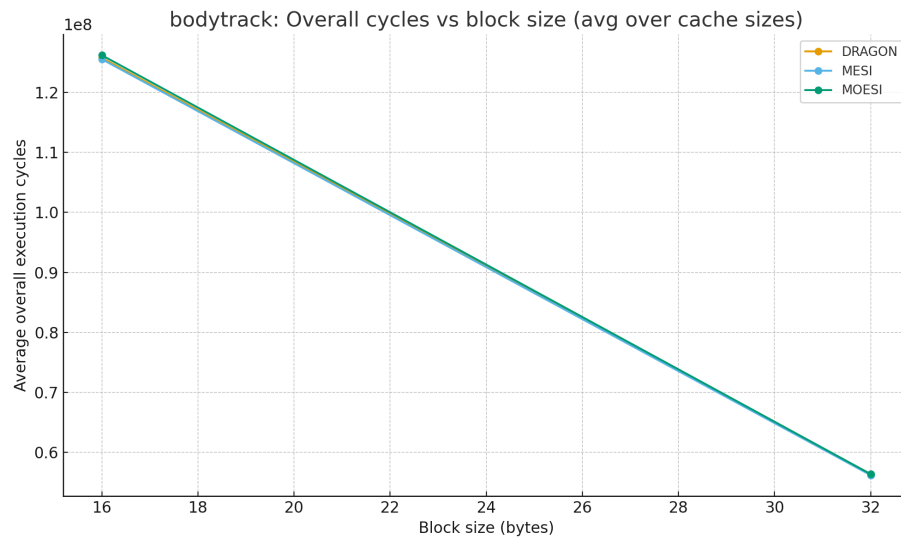
All three protocols benefit strongly as cache size increases from 1024B → 2048B → 4096B → 8192B, but the curves for MESI, MOESI and DRAGON almost overlap at each configuration. For example, in **blackscholes** the cycles drop by roughly an order of magnitude going from 1KB (270–271M cycles for MESI/MOESI, ~261M for DRAGON) to 8KB (~17–18M for all three), while bodytrack and fluidanimate show only about a 3× reduction over the same range. MOESI is numerically indistinguishable from MESI at every size, while DRAGON is at most a few percent faster (slightly better at the smallest cache for blackscholes). We therefore use **blackscholes** to illustrate the cache-size effect, because it is the most cache-sensitive workload with a smooth, monotonic drop in cycles, making it very clear that cache size dominates performance for all protocols and that the choice between MESI, MOESI and DRAGON has only a minor effect on execution time.

4 Block size effect on each protocol

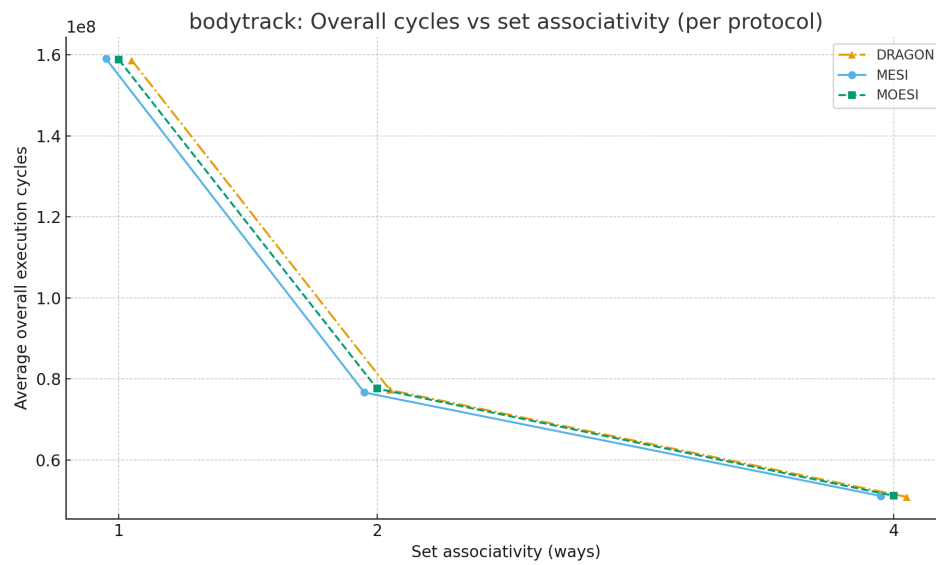
In our experiments(graph below), block size only varies between 16 B and 32 B, and these changes always occur together with an increase in cache capacity (1024/2048 B use 16 B blocks, 4096/8192 B use 32 B blocks). When we aggregate results by block size, all three benchmarks show a small but consistent reduction in overall cycles when moving from 16 B to 32 B blocks, and this trend is almost identical for MESI, MOESI and DRAGON. This suggests that in our configuration range, slightly larger blocks help by pulling in a bit more useful neighbouring data (some spatial locality) without yet suffering from having “too few” blocks. However, because block size and capacity are coupled in our setup, we cannot

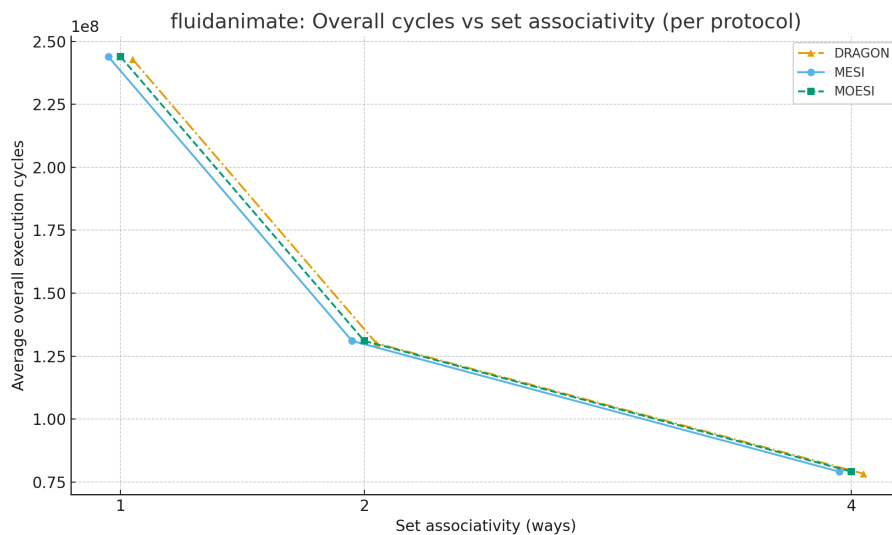
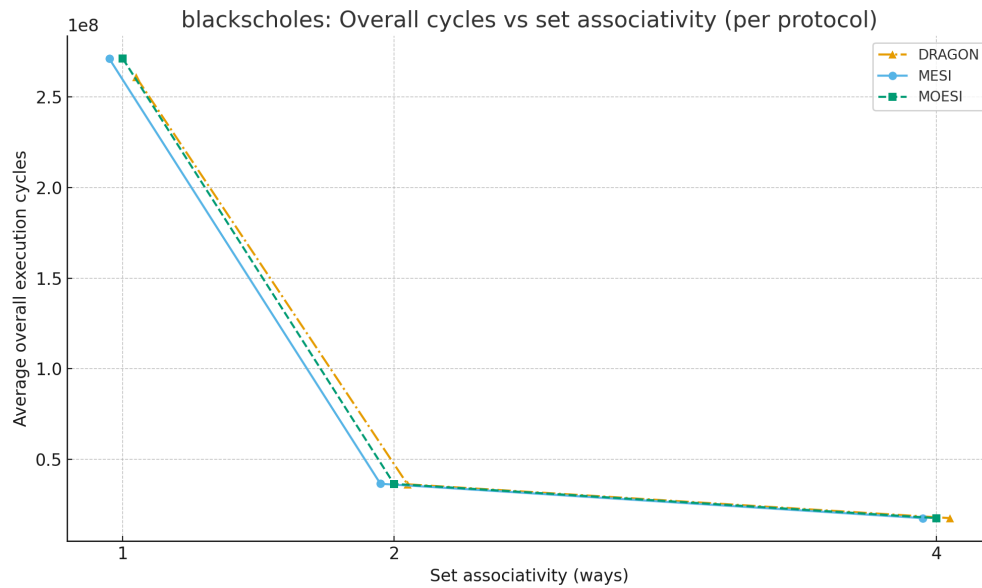
cleanly separate the benefit of larger blocks from the benefit of a larger cache; what we can say is that the 32 B configurations are uniformly better, and that block size does not favour any protocol in particular. MESI, MOESI and DRAGON all gain roughly the same small improvement.





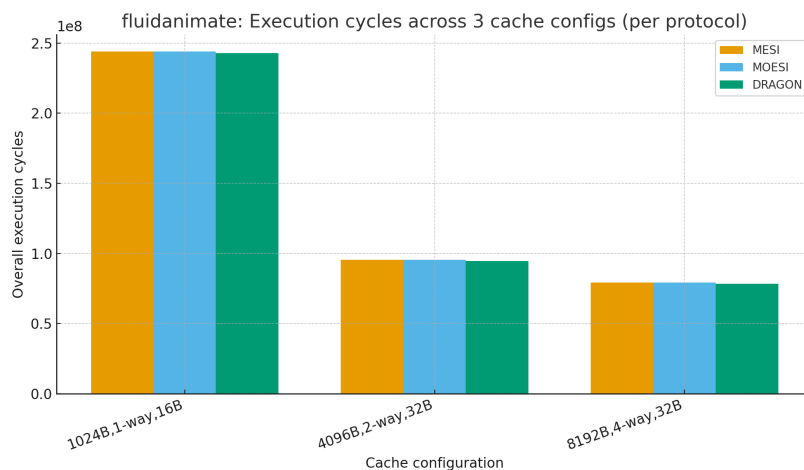
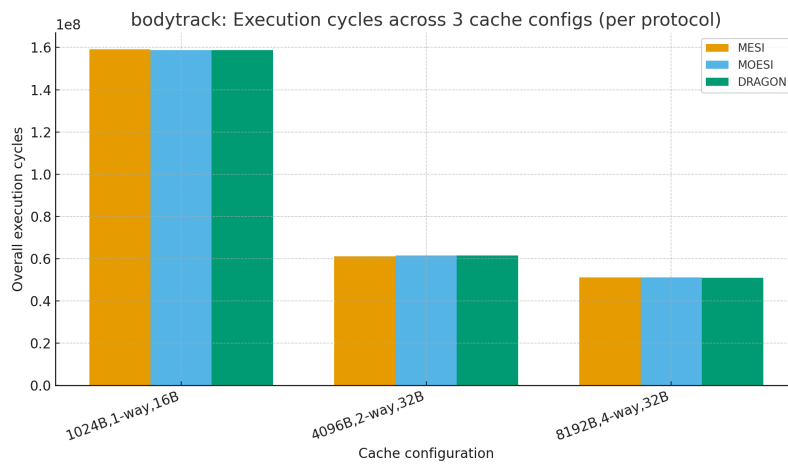
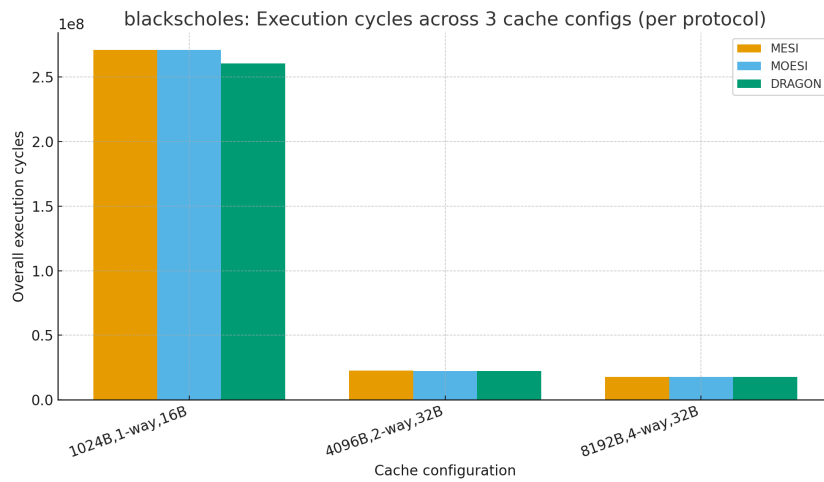
5 Set associativity effect on each protocol





Across our experiments (graph above), increasing set associativity from 1-way (direct-mapped) to 2-way and then 4-way gives a modest but consistent reduction in execution cycles, but this effect is much smaller than the effect of cache size and is almost identical across protocols. In the set-associativity plots, all three benchmarks show their highest average cycles at 1-way, with cycles dropping when we move to 2-way and then slightly again at 4-way, which indicates that extra ways do help by reducing conflict misses. However, the MESI, MOESI and DRAGON lines sit very close together at every associativity level, so no protocol gets a special advantage from higher associativity: it simply cleans up a few conflicts for everyone. Also, because associativity always increases together with cache capacity in our configurations, we cannot fully separate the two effects; what we can say is that higher associativity provides a small, protocol-agnostic improvement on top of the dominant benefit of more cache capacity.

6 Execution cycles comparison between protocols



For execution cycles, the grouped bars make the dominance of cache configuration even clearer: going from the smallest to the largest cache produces a large drop in cycle count for every benchmark, but within each of the three configurations the MESI, MOESI and DRAGON bars are nearly the same height. DRAGON tends to be slightly faster, and MOESI again tracks MESI very closely, yet these protocol differences are tiny compared with the big gains obtained by scaling cache size and associativity. Overall, the execution-cycle

comparison shows that all three protocols benefit in essentially the same way from better cache parameters, and protocol choice has only a minor impact on performance compared to the cache configuration itself.

7 Conclusion and result

Overall, Dragon and MESI give almost the same performance, but Dragon is cleaner in terms of coherence traffic. For all three benchmarks and all cache configurations, the execution cycles for Dragon and MESI are within a few percent of each other. All the big gains come from increasing cache size and associativity, not from switching protocol. However, for the same configuration Dragon usually has equal or slightly lower bus data traffic and fewer coherence actions. In other words: in these PARSEC workloads and cache sizes, Dragon does not buy a big speedup, but it achieves comparable performance with less coherence actions on the bus.

Compared to those two, MOESI is indistinguishable from MESI in your results. Across cache size sweeps, block sizes, and associativities, the MOESI curves for both execution cycles and bus traffic almost sit on top of MESI: same shape, same magnitude, no consistent advantage. The extra Owned state in MOESI does not translate into a measurable improvement for these particular benchmarks and configurations.

Conclusion

- MESI vs Dragon has almost the same performance, Dragon uses slightly less bus traffic and many fewer coherence events.
- MOESI vs MESI/Dragon: behaves like MESI with no visible gains, it neither significantly improves execution time nor reduces bus traffic compared to MESI, and still doesn't match Dragon's reduction in coherence activity.

Appendix

Running experience output, scripts and drawing scripts: [Running appendix](#).

High level implementation diagram code:

```
\documentclass{article}
```

```
\usepackage{tikz}
```

```
\usetikzlibrary{arrows.meta,positioning}
```

```
\begin{document}
```

```
\begin{figure}[h]
```

```
\centering
```

```
\begin{tikzpicture}[
```

```
font=\small,
```

```

    block/.style={rectangle,draw,rounded corners,minimum width=3.3cm,minimum
height=1.0cm,align=center},
    smallblock/.style={rectangle,draw,rounded corners,minimum width=2.6cm,minimum
height=0.8cm,align=center},
    proto/.style={rectangle,draw,dashed,rounded corners,minimum width=4.3cm,minimum
height=1.2cm,align=center},
    line/.style={-Latex,thick},
    dashedline/.style={-Latex,thick,dashed}
]

```

```

% === Right side: main + simulator + monitor (now further right) ===
\node[block] (main) at (10,0) {Main code\\(CacheCoherence.cpp)};
\node[block,below=1.4cm of main] (sim) {Operating\_System\\ / Simulator};
\node[block,below=1.4cm of sim] (monitor) {Monitor\\(statistics)};

```

```

\draw[line] (main.south) -- node[right]{\scriptsize 1.\ start simulation} (sim.north);
\draw[line] (sim.south) -- node[right]{\scriptsize 6.\ record stats} (monitor.north);

```

```

% === Left: cores & caches ===
\node[block] (bus) at (0,3) {Bus};

```

```

\node[block,below=1.0cm of bus] (p0) {Core 0};
\node[smallblock,below=0.25cm of p0] (c0) {L1 Cache 0};

```

```

\node[block,below=1.4cm of c0] (p1) {Core 1};
\node[smallblock,below=0.25cm of p1] (c1) {L1 Cache 1};

```

```

\node[block,below=1.4cm of c1] (p2) {Core 2};
\node[smallblock,below=0.25cm of p2] (c2) {L1 Cache 2};

```

```

\node[block,below=1.4cm of c2] (p3) {Core 3};
\node[smallblock,below=0.25cm of p3] (c3) {L1 Cache 3};

```

```

\node[block,below=1.5cm of c3] (mem) {Main Memory};

```

```

% core <-> cache links
\foreach \p/\c in {p0/c0, p1/c1, p2/c2, p3/c3} {
    \draw[line] (\p.south) -- (\c.north);
}

```

```

% bus connections (wider spacing left→middle)
\draw[line] (bus.south) -- ++(0,-0.3) -| (c0.west);
\draw[line] (bus.south) -- ++(0,-2.0) -| (c1.west);
\draw[line] (bus.south) -- ++(0,-3.7) -| (c2.west);
\draw[line] (bus.south) -- ++(0,-5.4) -| (c3.west);

```

```

% memory
\draw[line] (bus.south) -- ++(0,-1.0) -- (mem.north)

```

```

node[above] {\scriptsize 4a.\ block read/write};

% === Coherence Protocol Logic (moved further right for more width) ===
\node[proto,right=4.5cm of bus] (proto)
  {\Coherence Protocol Logic\\(MESI / Dragon / MOESI)};

% protocol → bus
\draw[dashedline] (proto.west) -- node[above]{\scriptsize manages snoop, ownership, state
machine} (bus.east);

% protocol → caches (positioned further right for spacing)
\draw[dashedline] (proto.south east) to[out=-60,in=90] node[right,pos=0.4]{\scriptsize state
updates} (c0.north east);
\draw[dashedline] (proto.south) to[out=-70,in=90] (c1.north east);
\draw[dashedline] (proto.south west) to[out=-80,in=90] (c2.north east);

% sim → cores
\draw[dashedline] (sim.west) -- node[above,sloped]{\scriptsize 2.\ pick core, decode trace}
++(-3.4,0)
  |- (p2.east);

% cache → monitor
\draw[dashedline] (c0.east) .. controls +(10,2) and +(-2,1.0) ..
  node[above]{\scriptsize 5.\ per-core stats update} (monitor.west);

% cache miss → bus
\draw[dashedline] (c2.south west) -- ++(-1.0,0) |-
  node[above,sloped]{\scriptsize 3.\ issue bus transaction} (bus.west);

% bus → other caches
\draw[dashedline] (bus.east) -- ++(2.5,0) |-
  node[above,sloped]{\scriptsize 4b.\ snoop / invalidate / update} (c1.north east);

% memory transfer highlight
% \draw[dashedline] (mem.east) -- ++(0,0) -- (c3.east)
%   node[below]{\scriptsize 4a.\ block read/write};

\end{tikzpicture}
\caption{High-level design of the cycle-accurate multi-core cache coherence simulator.}
\end{figure}

\end{document}

```

AI Declaration

ChatGPT 5 is used to help us clarify the understanding of cache protocols, write profiling scripts and help us refine wording and grammar in our reports.

